Reference Solutions — Assignment 4

Campus Bike-Share: Data Structures End to End

> ***This is one acceptable answer set, not an answer key.*** *The rubric is what decides a grade. Several questions here have more than one full-credit answer, and where that is true it is said so explicitly. A student answer that differs from this document is not thereby wrong — check it against the rubric criterion, which describes what an answer must demonstrate rather than what it must say.*

Q1. Choosing the right structure (12 points)

(a) A **hash table** (Python `dict`, or any open-addressed / chained table) keyed on `bikeID`.

The justification has to come from the access pattern. `lookup` dominates by orders of magnitude — thousands per hour against a handful of inserts per day and one report per night. A hash table gives expected O(1) lookup, which is the only operation whose cost is multiplied by a large constant here. Insert and remove are also expected O(1), and they are rare enough that their cost would not decide the choice either way.

The report is the one operation a hash table does badly: keys come out in no useful order, so the report costs O(n log n) to sort 40,000 ids. Once per night, that is roughly a millisecond of work. Paying it nightly to make the hot path O(1) is the right trade.

Also full credit: a balanced BST / order-statistic tree (`O(log n)` lookup, `O(n)` in-order report), **provided** the student argues the trade rather than asserting it — for instance that O(log n) on 40,000 keys is only ~16 comparisons and the ordered traversal comes free. The mark is for reasoning from the access pattern, not for picking the structure this document picked.

(b) No, the choice should not change, and the argument is arithmetic.

Sorting 40,000 keys is about 40,000 x log_2(40,000) ~ 40,000 x 15.3 ~ 6 x 10^5 comparisons, well under a second, and now paid 24 times a day instead of once — still negligible. Lookups, meanwhile, run thousands of times per hour. Moving to a tree to save the hourly sort would slow the dominant operation to buy back time on an operation that was never the bottleneck.

A student who argues the *opposite* — that at hourly frequency a structure maintaining sorted order starts to pay, especially if the report frequency keeps rising — earns full credit if the argument is quantitative rather than a preference. The failure mode this part is testing is answering "yes/no" without comparing costs.

(c) For a hash table:

operation	average case
`lookup(bikeID)`	O(1)
`insert(bikeID)`	O(1)
`remove(bikeID)`	O(1)
`report()`	O(n log n)

Noting that hash operations are O(n) in the worst case is correct and welcome, but the question asked for average case.

Q2. Hashing (14 points)

(a) Separate chaining stores colliding keys in a secondary structure — usually a linked list or small tree — hanging off the slot. **Open addressing** stores every key in the table itself, and on collision probes for another free slot by a fixed rule (linear, quadratic, double hashing).

- Advantage of chaining: it degrades gracefully. The load factor can exceed 1 without the table breaking, and deletion is a simple unlink.
- Advantage of open addressing: everything lives in one contiguous array, so it has no per-entry pointer overhead and far better cache locality — which in practice often makes it faster despite doing more probes.

(b) alpha = 12 / 16 = **0.75**.

Under linear probing the expected number of probes for an unsuccessful search is approximately ½(1 + 1/(1-alpha)²), which diverges as alpha -> 1. The mechanism matters more than the formula: as the table fills, occupied slots merge into long runs — **primary clustering** — and any key hashing anywhere into a run must walk to its end. Each insertion into a run also makes that run longer, so clusters grow super-linearly and performance collapses well before the table is actually full. At alpha = 1 the table is full and an unsuccessful search never terminates.

(c) Under open addressing, a key's position may be the *end of a probe chain* that passes through other slots. Simply emptying a slot breaks every chain that probed through it: a later search for a key stored beyond the hole stops at the hole and wrongly reports the key absent.

The standard fix is a **tombstone** — mark the slot deleted rather than empty. A search treats a tombstone as occupied and keeps probing; an insertion may reuse it. The cost is that tombstones count toward the effective load factor and are not reclaimed by deletion, so a table churned heavily fills with them and must periodically be **rehashed** to clear them out.

(d) `h(k) = k mod 7`, linear probing:

- `15 mod 7 = 1` -> slot 1 free -> **slot 1**
- `11 mod 7 = 4` -> slot 4 free -> **slot 4**
- `27 mod 7 = 6` -> slot 6 free -> **slot 6**
- `8 mod 7 = 1` -> slot 1 holds 15 -> probe slot 2, free -> **slot 2**
- `22 mod 7 = 1` -> slot 1 holds 15 -> slot 2 holds 8 -> probe slot 3, free -> **slot 3**

Final table:

```
index:  0     1     2     3     4     5     6
value:  -    15     8    22    11     -    27
```

Q3. The k nearest stations (18 points)

(a)

```
import heapq

def k_closest_stations(stations, user, k):
    if k <= 0:
        return []

    ux, uy = user
    heap = []  # max-heap of (-squared_distance, station_id), size <= k

    for s in stations:
        d = (s.x - ux) ** 2 + (s.y - uy) ** 2
        if len(heap) < k:
            heapq.heappush(heap, (-d, s.id))
        elif d < -heap[0][0]:
            heapq.heapreplace(heap, (-d, s.id))

    # Farthest sits at the root, so sort by distance ascending on the way out.
    return [sid for _, sid in sorted(heap, key=lambda pair: -pair[0])]
```

Two details worth the marks: **squared** distance is compared rather than the true distance, because the square root is monotonic and computing it n times is wasted work; and Python's `heapq` is a min-heap, so distances are negated to make it behave as the max-heap the algorithm needs.

`k <= 0` returns empty. `k > n` needs no special case — the heap simply never fills, and all `n` stations are returned.

(b) O(n log k).

Each of the `n` stations is examined once. For each, the work is either a push into a heap of size at most `k`, or a comparison against the root followed by at most one `heapreplace`. Both heap operations are O(log k) because the heap never exceeds `k` elements — that bound is the whole point of evicting the farthest rather than accumulating. The root comparison itself is O(1), so stations that are not among the closest so far cost nothing beyond it. The final sort is O(k log k), which is dominated by O(n log k) whenever k <= n.

Space is O(k).

(c) Sorting everything costs O(n log n) time and O(n) extra space. With n = 40,000 and k = 5, log_2 n ~ 15.3 against log_2 k ~ 2.3 — roughly a 6.6x reduction in the per-element factor, and the heap holds 5 entries instead of 40,000. The saving comes from never ordering the 39,995 stations the rider will never see.

The advantage disappears as **k approaches n**: at k = n, log k = log n and the two are asymptotically identical, with the heap now carrying the larger constant factor and no memory advantage. In practice sorting wins earlier than that — once k is a substantial fraction of n, a well-optimised sort beats the heap's per-element overhead.

Q4. Shortest walking route (20 points)

(a)

```
import heapq
from math import inf
```

```
def shortest_route(graph, source, target):
    if source not in graph or target not in graph:
        return (inf, [])
    if source == target:
        return (0, [source])

    dist = {source: 0}
    parent = {}
    settled = set()
    heap = [(0, source)]

    while heap:
        d, node = heapq.heappop(heap)
        if node in settled:
            continue           # a stale entry left by a decrease-key
        settled.add(node)

        if node == target:
            path = [node]
            while path[-1] != source:
                path.append(parent[path[-1]])
            return (d, path[::-1])

        for neighbour, weight in graph.get(node, ()):
            if neighbour in settled:
                continue
            candidate = d + weight
            if candidate < dist.get(neighbour, inf):
                dist[neighbour] = candidate
                parent[neighbour] = node
                heapq.heappush(heap, (candidate, neighbour))

    return (inf, [])
```

The `settled` check on pop is what makes this correct with a binary heap that has no decrease-key: a better distance to a node is pushed as a *new* entry, so the heap can hold several entries per node and the stale ones must be skipped when popped. `parent` records the predecessor on the best known path, which is what allows the path to be rebuilt backwards from the target.

(b) O((V + E) log V), which simplifies to O(E log V) on a connected graph.

Term by term:

- Every vertex is settled at most once, giving V pops. Each pop is O(log V). -> **O(V log V)**
- Each edge is relaxed at most once, from its already-settled endpoint. A successful relaxation pushes one entry, costing O(log V). Because stale entries are never removed, the heap holds up to O(E) entries, so a pop is strictly O(log E) — but $E <= V^2$, so log E <= 2 log V, and the term stays O(log V). -> **O(E log V)**
- Building the initial structures is O(V), and reconstructing the path walks at most V parents, O(V). Both are dominated.

Total: O(V log V + E log V) = **O((V + E) log V)**.

Full credit also for a student who assumes a decrease-key heap and derives O(V log V + E), or who derives $O(V^2)$ for the array-based version, provided the derivation matches the implementation they actually wrote.

(c) Three nodes, directed edges:

```
A -> B   weight  2
A -> C   weight  5
C -> B   weight -4
```

Dijkstra from A settles B first at distance 2, because 2 is the smallest tentative distance on the heap. Once B is settled it is never revisited, so the algorithm returns **dist(B) = 2**.

The true shortest path is A -> C -> B, costing 5 + (-4) = **1**.

The violated assumption: Dijkstra settles the nearest unsettled vertex and treats that distance as final, which is only sound if extending a path can never reduce its cost — that is, if all edge weights are non-negative. A negative edge lets a longer prefix lead to a cheaper total, so a vertex settled early can still be improved later.

Q5. Amortized analysis (12 points)

(a) Consider `n` appends into an array that starts at capacity 1 and doubles when full. Each append does O(1) work of its own, so the only question is the total cost of copying.

Resizes happen when the array is at capacity 1, 2, 4, 8, …, and a resize at capacity `c` copies `c` elements. Across `n` appends the last resize is at capacity <= n, so total copy cost is:

```
1 + 2 + 4 + ... + 2^floor(log2(n))  =  2^(floor(log2 n) + 1) - 1  <  2n
```

This is a geometric series with ratio 2; its sum is bounded by twice its largest term. Total work for `n` appends is therefore at most `n + 2n = 3n` = O(n), so the amortized cost per append is **O(n)/n = O(1)**.

The intuition behind the arithmetic: each doubling is twice as expensive as the last but happens half as often, so the expensive events are spread thinly enough that their total is still linear.

(b) Growing by a fixed +10 resizes at sizes 10, 20, 30, …, n, copying 10, 20, 30, … elements. With `n/10` resizes:

```
10 + 20 + ... + n  =  10 x (1 + 2 + ... + n/10)
                   =  10 x (n/10)(n/10 + 1)/2
                   ~  n² / 20
```

Total copy cost is **Theta(n^2)**, so the amortized cost per append is **Theta(n)** — not constant.

It is a bad trade. The memory saving is bounded and small: doubling wastes at most 2x the live data, while +10 growth wastes at most 10 slots, so the most the proposal can save is a constant factor of memory. What it costs is a change in *asymptotic time* — appending 1,000,000 elements goes from about 2,000,000 copy operations to about 50,000,000,000. Trading a constant factor of space for a linear factor of time is the wrong direction.

The general result is that **any** geometric growth factor r > 1 gives amortized O(1), with the constant trading off against wasted memory; arithmetic growth gives amortized Theta(n) for any fixed increment.

Q6. Debugging (12 points)

(a) Two distinct faults.

Fault 1 — `hi` is initialised past the end (line 2), combined with `lo <= hi` (line 3). Setting `hi = len(sorted_ids)` makes the last valid index `len - 1`, but the loop condition allows `lo == hi == len`, so `mid` can equal `len` and `sorted_ids[mid]` raises `IndexError`. The half-open initialisation `hi = len` is only correct with a half-open condition `while lo < hi`; mixing it with the inclusive `<=` is the error.

Fault 2 — `lo = mid` instead of `lo = mid + 1` (line 7). When the midpoint is too small, the correct move is to exclude it. Assigning `lo = mid` leaves it in the range, and once `hi == lo + 1`, `mid = (lo + hi) // 2` floors back to `lo`, so `lo` never advances and the loop **spins forever**. The search interval stops shrinking, which is the property the algorithm's termination depends on.

(b) Fault 1 is the hidden one. Fault 2 masks it completely: no input exposes the out-of-range read while `lo = mid` is still there.

Why: reaching `mid = len(sorted_ids)` requires `lo + hi >= 2·len`, and since `hi <= len`, that means `lo` must reach `len` too. But `lo` only ever advances via `lo = mid`, and `mid = (lo + hi) // 2 < hi <= len` whenever `lo < hi` — so `lo` can never climb to `len`. Instead it stalls as soon as `hi == lo + 1`, and the loop spins. **Every input that would have reached the bad index hangs first.**

- **The failure you can observe (fault 2):** `find_station([10, 20, 30], 25)`. At `lo = 1, hi = 2`, `mid = 1`, `20 < 25`, so `lo = 1` again — unchanged. The function **hangs**, never returning. Any absent target above the first element does the same.
- **The input that exposes fault 1 once fault 2 is fixed:** repair line 7 to `lo = mid + 1` and leave `hi = len(sorted_ids)`, then call `find_station([10, 20, 30], 35)`. Now `lo` advances properly: `mid = 1` -> `lo = 2`, `mid = 2` -> `lo = 3`, `mid = 3`, and `sorted_ids[3]` raises `IndexError: list index out of range`. Any absent target greater than every element does it.

This is the point of the question: fixing the visible bug *reveals* the second one rather than finishing the job, and a student who tests only until the hang goes away ships the crash.

(c) Corrected:

```
def find_station(sorted_ids, target):
    lo, hi = 0, len(sorted_ids) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if sorted_ids[mid] == target:
            return mid
        elif sorted_ids[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

The half-open form is equally correct:

```
def find_station(sorted_ids, target):
    lo, hi = 0, len(sorted_ids)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_ids[mid] == target:
            return mid
        elif sorted_ids[mid] < target:
            lo = mid + 1
        else:
```

```
            hi = mid
    return -1
```

What both share is the invariant the original broke: every iteration must strictly shrink the interval, and the interval must only ever contain valid indices.

Q7. Designing a spatial query (12 points)

Both recommendations below are defensible. The marks are for the comparison and the argument, not for the choice.

Approach 1 — linear scan. Test all 40,000 bikes against the radius on every query.

- Query: **O(n)**, 40,000 distance tests. At 500 queries/second that is 2 x 10^7 distance tests per second — feasible on one core with squared distances, but it is the entire budget with nothing left over, and it scales linearly with fleet size.
- Movement: **O(1)**, free. Nothing is indexed, so nothing needs maintaining.
- Memory: **zero** overhead beyond the bike positions themselves.

Approach 2 — uniform grid. Divide the 5 km campus into square cells and keep a bucket of bike ids per cell. A query touches only the cells overlapping the query circle.

- Query: **O(C + m)**, where C is the number of cells overlapping the circle and m the bikes inside them. With 500 m cells the campus is a 10 x 10 grid of 100 cells, and a 2 km radius covers roughly $\pi(2000)^2/(500)^2 \sim 50$ cells — **half the campus**. Against 40,000 bikes over 100 cells, that is ~20,000 bikes examined: a 2x saving, not an order of magnitude.
- Movement: **O(1)** expected — a move is a bucket removal and insertion, and most moves stay within a cell. But it is 40,000 constant-time updates that the linear scan does not pay at all.
- Memory: O(number of cells + n) for the buckets, plus a cell id per bike.

Recommendation: the linear scan.

The decisive fact is the ratio of the query radius to the search space. A 2 km radius on a 5 km campus selects roughly half of it, and **no spatial index helps when the query returns half the data** — indexes earn their keep by pruning, and there is almost nothing here to prune. The grid adds per-move bookkeeping and memory to buy a factor of about two on a scan that a single core already handles, and a scan of a contiguous array is close to the best case for the cache, so the real-world factor will be smaller than the analysis suggests.

The recommendation flips under conditions worth naming: if the radius drops to 100–200 m, if the fleet grows past a few hundred thousand, or if the campus extends to a whole city, the pruning becomes real and a grid or k-d tree wins. A k-d tree specifically is a poor fit here regardless of scale, because the positions change constantly and it rebalances badly under motion — that is the axis on which grids beat trees for moving objects.

A recommendation of the grid also earns full credit if it is argued — for example on the grounds that 500 queries/second leaves no headroom for growth, that the constant-time move cost is genuinely negligible, or that cell size should be tuned to the radius rather than fixed at 500 m. What does not earn the marks is naming a structure without comparing it to the alternative on this workload.